Task 4: SQL-Based Machine Learning on the World Happiness Dataset

This notebook implements the required SQL-based machine learning tasks on the World Happiness dataset.

1. Multivariate linear regression (`Score` as target)
2. Multi-level decision tree classification (`Score` into `high/middle/low`)
3. K-means clustering (`k=3`) with min-max normalization

All model computations are expressed with SQL queries; Python is used only to orchestrate execution and display results.

In [1]:
```
import sqlite3
import math
from pathlib import Path
import pandas as pd
from IPython.display import display

pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 180)


def find_world_happiness_dir(start: Path) -> Path:
    candidates = [start] + list(start.parents)
    for root in candidates:
        p = root / 'world_happiness'
        if p.exists() and p.is_dir():
            return p
    raise FileNotFoundError('Could not locate world_happiness directory from cur


current_dir = Path.cwd()
world_happiness_dir = find_world_happiness_dir(current_dir)
out_db_path = current_dir / 'world_happiness_task4.db'

print(f'Current directory: {current_dir}')
print(f'Dataset directory: {world_happiness_dir}')
print(f'SQLite database: {out_db_path}')
```

```
Current directory: D:\Courses\sixth_semester\Database\lab\lab3\task4
Dataset directory: D:\Courses\sixth_semester\Database\lab\lab3\task4\world_happin
ess
SQLite database: D:\Courses\sixth_semester\Database\lab\lab3\task4\world_happines
s_task4.db
```

In [2]:
```
# 1) Load and harmonize 2015-2019 CSV files into a unified schema.
schema_columns = [
    'Overall_rank',
    'Country',
    'Score',
    'GDP_per_capita',
    'Social_support',
    'Healthy_life_expectancy',
```

```
    'Freedom_to_make_life_choices',
    'Generosity',
    'Perceptions_of_corruption',
    'Year'
]

column_maps = {
    '2015': {
        'Happiness Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness Score': 'Score',
        'Economy (GDP per Capita)': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health (Life Expectancy)': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust (Government Corruption)': 'Perceptions_of_corruption',
    },
    '2016': {
        'Happiness Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness Score': 'Score',
        'Economy (GDP per Capita)': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health (Life Expectancy)': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust (Government Corruption)': 'Perceptions_of_corruption',
    },
    '2017': {
        'Happiness.Rank': 'Overall_rank',
        'Country': 'Country',
        'Happiness.Score': 'Score',
        'Economy..GDP.per.Capita.': 'GDP_per_capita',
        'Family': 'Social_support',
        'Health..Life.Expectancy.': 'Healthy_life_expectancy',
        'Freedom': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Trust..Government.Corruption.': 'Perceptions_of_corruption',
    },
    '2018': {
        'Overall rank': 'Overall_rank',
        'Country or region': 'Country',
        'Score': 'Score',
        'GDP per capita': 'GDP_per_capita',
        'Social support': 'Social_support',
        'Healthy life expectancy': 'Healthy_life_expectancy',
        'Freedom to make life choices': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
        'Perceptions of corruption': 'Perceptions_of_corruption',
    },
    '2019': {
        'Overall rank': 'Overall_rank',
        'Country or region': 'Country',
        'Score': 'Score',
        'GDP per capita': 'GDP_per_capita',
        'Social support': 'Social_support',
        'Healthy life expectancy': 'Healthy_life_expectancy',
        'Freedom to make life choices': 'Freedom_to_make_life_choices',
        'Generosity': 'Generosity',
```

```
        'Perceptions of corruption': 'Perceptions_of_corruption',
    }
}

frames = []
for csv_path in sorted(world_happiness_dir.glob('*.csv')):
    year = csv_path.stem
    if year not in column_maps:
        continue
    df = pd.read_csv(csv_path)
    rename_map = column_maps[year]
    selected = df[list(rename_map.keys())].rename(columns=rename_map).copy()
    selected['Year'] = int(year)
    frames.append(selected)

happiness_df = pd.concat(frames, ignore_index=True)

for col in schema_columns:
    if col not in happiness_df.columns:
        happiness_df[col] = None

numeric_cols = [c for c in schema_columns if c not in ('Country',)]
for c in numeric_cols:
    happiness_df[c] = pd.to_numeric(happiness_df[c], errors='coerce')

happiness_df = happiness_df[schema_columns].dropna(subset=[
    'Overall_rank', 'Country', 'Score', 'GDP_per_capita', 'Social_support',
    'Healthy_life_expectancy', 'Freedom_to_make_life_choices', 'Generosity',
    'Perceptions_of_corruption'
]).reset_index(drop=True)

print('Unified dataset shape:', happiness_df.shape)
display(happiness_df.head())
```

```
Unified dataset shape: (781, 10)
```

	Overall_rank	Country	Score	GDP_per_capita	Social_support	Healthy_life_exp
0	1	Switzerland	7.587	1.39651	1.34951	
1	2	Iceland	7.561	1.30232	1.40223	
2	3	Denmark	7.527	1.32548	1.36058	
3	4	Norway	7.522	1.45900	1.33095	
4	5	Canada	7.427	1.32629	1.32261	

In [3]:
```
# 2) Build SQLite table: happyness
conn = sqlite3.connect(out_db_path)
conn.create_function('SQRT', 1, lambda x: None if x is None else math.sqrt(x))
conn.create_function('POWER', 2, lambda x, y: None if x is None or y is None els
conn.create_function('LOG', 1, lambda x: None if x is None or float(x) <= 0 else
cur = conn.cursor()

cur.executescript('''
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS happyness;
CREATE TABLE happyness (
```

```
    Overall_rank INTEGER,
    Country TEXT,
    Score REAL,
    GDP_per_capita REAL,
    Social_support REAL,
    Healthy_life_expectancy REAL,
    Freedom_to_make_life_choices REAL,
    Generosity REAL,
    Perceptions_of_corruption REAL,
    Year INTEGER
);
''')

happiness_df.to_sql('happyness', conn, if_exists='append', index=False)

cur.executescript('''
CREATE INDEX IF NOT EXISTS idx_happyness_country ON happyness(Country);
CREATE INDEX IF NOT EXISTS idx_happyness_score ON happyness(Score);
''')

conn.commit()

profile_df = pd.read_sql_query('''
SELECT
    COUNT(*) AS n_rows,
    COUNT(DISTINCT Country) AS n_countries,
    MIN(Score) AS min_score,
    MAX(Score) AS max_score,
    AVG(Score) AS avg_score
FROM happyness;
''', conn)
display(profile_df)

sample_df = pd.read_sql_query('SELECT * FROM happyness ORDER BY Year, Overall_ra
display(sample_df)
```

	n_rows	n_countries	min_score	max_score	avg_score
0	781	170	2.693	7.769	5.377232

	Overall_rank	Country	Score	GDP_per_capita	Social_support	Healthy_life_exp
0	1	Switzerland	7.587	1.39651	1.34951	
1	2	Iceland	7.561	1.30232	1.40223	
2	3	Denmark	7.527	1.32548	1.36058	
3	4	Norway	7.522	1.45900	1.33095	
4	5	Canada	7.427	1.32629	1.32261	
5	6	Finland	7.406	1.29025	1.31826	
6	7	Netherlands	7.378	1.32944	1.28017	
7	8	Sweden	7.364	1.33171	1.28907	
8	9	New Zealand	7.286	1.25018	1.31967	
9	10	Australia	7.284	1.33358	1.30923	

A) Multivariate Linear Regression (SQL)

Target variable: `Score`

Features:

- `GDP_per_capita`
- `Social_support`
- `Healthy_life_expectancy`
- `Freedom_to_make_life_choices`
- `Perceptions_of_corruption`

Approach:

1. Min-max normalize features with SQL
2. Train weights with gradient descent where each gradient is computed by SQL
3. Evaluate regression losses in SQL

In [4]:
```
# Prepare normalized training table with SQL.
cur.executescript('''
DROP TABLE IF EXISTS regression_data;
CREATE TABLE regression_data AS
WITH stats AS (
    SELECT
        MIN(GDP_per_capita) AS min_gdp,
        MAX(GDP_per_capita) AS max_gdp,
        MIN(Social_support) AS min_social,
        MAX(Social_support) AS max_social,
        MIN(Healthy_life_expectancy) AS min_health,
        MAX(Healthy_life_expectancy) AS max_health,
        MIN(Freedom_to_make_life_choices) AS min_freedom,
        MAX(Freedom_to_make_life_choices) AS max_freedom,
        MIN(Perceptions_of_corruption) AS min_corr,
```

```
        MAX(Perceptions_of_corruption) AS max_corr
    FROM happyness
)
SELECT
    rowid AS sample_id,
    Score AS y,
    (GDP_per_capita - min_gdp) / NULLIF(max_gdp - min_gdp, 0) AS x1,
    (Social_support - min_social) / NULLIF(max_social - min_social, 0) AS x2,
    (Healthy_life_expectancy - min_health) / NULLIF(max_health - min_health, 0)
    (Freedom_to_make_life_choices - min_freedom) / NULLIF(max_freedom - min_free
    (Perceptions_of_corruption - min_corr) / NULLIF(max_corr - min_corr, 0) AS x
FROM happyness, stats;

DROP TABLE IF EXISTS lr_weights;
CREATE TABLE lr_weights (
    b0 REAL,
    b1 REAL,
    b2 REAL,
    b3 REAL,
    b4 REAL,
    b5 REAL
);
INSERT INTO lr_weights VALUES (0.0, 0.0, 0.0, 0.0, 0.0, 0.0);

DROP TABLE IF EXISTS lr_training_log;
CREATE TABLE lr_training_log (
    iter INTEGER,
    mse REAL
);
''')
conn.commit()

learning_rate = 0.08
iterations = 500

for i in range(iterations):
    grads = pd.read_sql_query('''
        WITH w AS (
            SELECT b0, b1, b2, b3, b4, b5 FROM lr_weights LIMIT 1
        )
        SELECT
            AVG(-2.0 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5))) AS g0
            AVG(-2.0 * x1 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x2 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x3 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x4 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
            AVG(-2.0 * x5 * (y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5)))
        FROM regression_data, w;
    ''', conn).iloc[0]

    cur.execute('''
        UPDATE lr_weights
        SET
            b0 = b0 - ?,
            b1 = b1 - ?,
            b2 = b2 - ?,
            b3 = b3 - ?,
            b4 = b4 - ?,
            b5 = b5 - ?;
    ''', tuple(learning_rate * float(grads[f'g{k}']) for k in range(6)))
```

```
    if i % 20 == 0 or i == iterations - 1:
        mse_value = pd.read_sql_query('''
            WITH w AS (SELECT * FROM lr_weights LIMIT 1)
            SELECT AVG(POWER(y - (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5), 2
            FROM regression_data, w;
        ''', conn)['mse'].iloc[0]
        cur.execute('INSERT INTO lr_training_log(iter, mse) VALUES (?, ?);', (i,

conn.commit()

weights_df = pd.read_sql_query('SELECT * FROM lr_weights;', conn)
loss_curve_df = pd.read_sql_query('SELECT * FROM lr_training_log ORDER BY iter;'

display(weights_df)
display(loss_curve_df.tail(10))
```

	b0	b1	b2	b3	b4	b5
0	2.225761	1.78704	1.165615	1.352721	1.136308	0.619361

	iter	mse
16	320	0.307263
17	340	0.307012
18	360	0.306786
19	380	0.306582
20	400	0.306397
21	420	0.306229
22	440	0.306075
23	460	0.305935
24	480	0.305807
25	499	0.305695

In [5]:
```
# Create prediction table and compute regression loss functions in SQL.
cur.executescript('''
DROP VIEW IF EXISTS lr_predictions;
CREATE VIEW lr_predictions AS
WITH w AS (SELECT * FROM lr_weights LIMIT 1)
SELECT
    d.sample_id,
    d.y AS y_true,
    (b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + b5*x5) AS y_pred
FROM regression_data d, w;
''')
conn.commit()

regression_losses = pd.read_sql_query('''
SELECT
    AVG(POWER(y_true - y_pred, 2)) AS mse,
    SQRT(AVG(POWER(y_true - y_pred, 2))) AS rmse,
    AVG(ABS(y_true - y_pred)) AS mae,
    AVG(ABS((y_true - y_pred) / NULLIF(y_true, 0.0))) * 100.0 AS mape,
```

```
    AVG(
        CASE
            WHEN ABS(y_true - y_pred) <= 1.0 THEN 0.5 * POWER(y_true - y_pred, 2
            ELSE ABS(y_true - y_pred) - 0.5
        END
    ) AS huber_loss
FROM lr_predictions;
''', conn)

display(regression_losses)

df_pred_preview = pd.read_sql_query('''
SELECT * FROM lr_predictions
ORDER BY ABS(y_true - y_pred) DESC
LIMIT 10;
''', conn)
display(df_pred_preview)
```

	mse	rmse	mae	mape	huber_loss
0	0.305695	0.552897	0.430378	8.521703	0.147942

	sample_id	y_true	y_pred
0	773	3.488	5.496157
1	615	3.590	5.501450
2	777	3.334	5.133028
3	620	3.408	5.148856
4	457	3.766	5.472434
5	234	5.440	3.841046
6	466	3.471	5.059566
7	154	3.465	5.043154
8	156	3.006	4.510196
9	778	3.231	4.718967

B) Multi-Level Decision Tree Classification (SQL + orchestration)

Task setup:

- Convert `Score` into three classes (`high` , `middle` , `low`)
- Use SQL to search split candidates and minimize weighted Gini impurity
- Grow a multi-level tree with anti-overfitting constraints:
  - `max_depth = 3`
  - `min_samples_split`
  - `min_samples_leaf`

In [6]:
```
# 1) Build classification dataset with three classes by score tertiles.
cur.executescript('''
```

```
DROP TABLE IF EXISTS dt_dataset;
CREATE TABLE dt_dataset AS
WITH ranked AS (
    SELECT
        rowid AS sample_id,
        Country,
        Score,
        GDP_per_capita,
        Social_support,
        Healthy_life_expectancy,
        Freedom_to_make_life_choices,
        Generosity,
        Perceptions_of_corruption,
        NTILE(3) OVER (ORDER BY Score DESC, Country ASC) AS tertile_bucket
    FROM happyness
)
SELECT
    sample_id,
    Country,
    Score,
    GDP_per_capita,
    Social_support,
    Healthy_life_expectancy,
    Freedom_to_make_life_choices,
    Generosity,
    Perceptions_of_corruption,
    CASE
        WHEN tertile_bucket = 1 THEN 'high'
        WHEN tertile_bucket = 2 THEN 'middle'
        ELSE 'low'
    END AS score_class
FROM ranked;

DROP TABLE IF EXISTS dt_long;
CREATE TABLE dt_long AS
SELECT sample_id, score_class, 'GDP_per_capita' AS feature, GDP_per_capita AS fe
UNION ALL
SELECT sample_id, score_class, 'Social_support' AS feature, Social_support AS fe
UNION ALL
SELECT sample_id, score_class, 'Healthy_life_expectancy' AS feature, Healthy_lif
UNION ALL
SELECT sample_id, score_class, 'Freedom_to_make_life_choices' AS feature, Freedo
UNION ALL
SELECT sample_id, score_class, 'Generosity' AS feature, Generosity AS feature_va
UNION ALL
SELECT sample_id, score_class, 'Perceptions_of_corruption' AS feature, Perceptio
''')
conn.commit()

class_dist = pd.read_sql_query('''
SELECT score_class, COUNT(*) AS n
FROM dt_dataset
GROUP BY score_class
ORDER BY n DESC;
''', conn)

display(class_dist)
```

	score_class	n
0	high	261
1	middle	260
2	low	260

In [7]:
```
# 2) Train a multi-level decision tree with regularization constraints.
max_depth = 3
min_samples_split = 30
min_samples_leaf = 12
candidate_deciles = 10
min_impurity_decrease = 1e-4

cur.executescript('''
DROP TABLE IF EXISTS dt_nodes;
CREATE TABLE dt_nodes (
    node_id INTEGER PRIMARY KEY,
    depth INTEGER NOT NULL,
    parent_id INTEGER,
    branch_from_parent TEXT,
    is_leaf INTEGER NOT NULL,
    split_feature TEXT,
    split_threshold REAL,
    predicted_class TEXT,
    n_samples INTEGER,
    gini REAL
);

DROP TABLE IF EXISTS dt_node_membership;
CREATE TABLE dt_node_membership (
    sample_id INTEGER PRIMARY KEY,
    node_id INTEGER NOT NULL
);
''')

cur.execute('INSERT INTO dt_node_membership(sample_id, node_id) SELECT sample_id


def compute_node_stats(node_id: int):
    counts = pd.read_sql_query('''
        SELECT d.score_class, COUNT(*) AS n
        FROM dt_node_membership m
        JOIN dt_dataset d ON d.sample_id = m.sample_id
        WHERE m.node_id = ?
        GROUP BY d.score_class
        ORDER BY n DESC, d.score_class ASC;
    ''', conn, params=(int(node_id),))

    n_samples = int(counts['n'].sum()) if not counts.empty else 0
    if n_samples == 0:
        return {'n_samples': 0, 'majority_class': 'middle', 'gini': 0.0}

    probs = counts['n'] / n_samples
    gini = float(1.0 - (probs * probs).sum())
    majority_class = str(counts.iloc[0]['score_class'])
    return {'n_samples': n_samples, 'majority_class': majority_class, 'gini': gi
```

```
root_stats = compute_node_stats(1)
cur.execute('''
    INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent, is_leaf,
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
''', (1, 0, None, None, 1, None, None, root_stats['majority_class'], root_stats[

next_node_id = 1
split_logs = []

for depth in range(max_depth):
    frontier = pd.read_sql_query('''
        SELECT node_id
        FROM dt_nodes
        WHERE depth = ? AND is_leaf = 1
        ORDER BY node_id;
    ''', conn, params=(depth,))

    if frontier.empty:
        break

    for node_id in frontier['node_id'].tolist():
        node_id = int(node_id)
        node_info = compute_node_stats(node_id)

        if node_info['n_samples'] < min_samples_split:
            continue
        if node_info['gini'] <= 1e-12:
            continue

        best_split = pd.read_sql_query('''
            WITH
            node_samples AS (
                SELECT sample_id
                FROM dt_node_membership
                WHERE node_id = ?
            ),
            candidate_points AS (
                SELECT DISTINCT feature, feature_value AS threshold
                FROM (
                    SELECT
                        l.feature,
                        l.feature_value,
                        NTILE(?) OVER (PARTITION BY l.feature ORDER BY l.feature
                    FROM dt_long l
                    JOIN node_samples ns ON ns.sample_id = l.sample_id
                ) t
                WHERE bucket_id BETWEEN 2 AND ? - 1
            ),
            assignments AS (
                SELECT
                    c.feature,
                    c.threshold,
                    l.score_class,
                    CASE WHEN l.feature_value <= c.threshold THEN 'L' ELSE 'R' E
                FROM candidate_points c
                JOIN dt_long l ON l.feature = c.feature
                JOIN node_samples ns ON ns.sample_id = l.sample_id
            ),
            side_class_counts AS (
```

```
    SELECT feature, threshold, side, score_class, COUNT(*) AS n
    FROM assignments
    GROUP BY feature, threshold, side, score_class
),
side_totals AS (
    SELECT
        feature,
        threshold,
        SUM(CASE WHEN side = 'L' THEN n ELSE 0 END) AS n_left,
        SUM(CASE WHEN side = 'R' THEN n ELSE 0 END) AS n_right
    FROM side_class_counts
    GROUP BY feature, threshold
),
valid_splits AS (
    SELECT feature, threshold, n_left, n_right
    FROM side_totals
    WHERE n_left >= ? AND n_right >= ?
),
proportions AS (
    SELECT
        scc.feature,
        scc.threshold,
        scc.side,
        scc.score_class,
        scc.n,
        CASE
            WHEN scc.side = 'L' THEN vs.n_left
            ELSE vs.n_right
        END AS side_n,
        1.0 * scc.n / CASE WHEN scc.side = 'L' THEN vs.n_left ELSE v
    FROM side_class_counts scc
    JOIN valid_splits vs
      ON scc.feature = vs.feature
     AND scc.threshold = vs.threshold
),
side_impurity AS (
    SELECT
        feature,
        threshold,
        side,
        MAX(side_n) AS side_n,
        1.0 - SUM(p * p) AS gini
    FROM proportions
    GROUP BY feature, threshold, side
),
node_total AS (
    SELECT COUNT(*) AS total_n FROM node_samples
),
split_scores AS (
    SELECT
        si.feature,
        si.threshold,
        SUM((1.0 * si.side_n / node_total.total_n) * si.gini) AS wei
    FROM side_impurity si
    CROSS JOIN node_total
    GROUP BY si.feature, si.threshold
)
SELECT feature, threshold, weighted_gini
FROM split_scores
ORDER BY weighted_gini ASC, feature ASC, threshold ASC
```

```
            LIMIT 1;
        ''', conn, params=(node_id, candidate_deciles, candidate_deciles, min_sa

        if best_split.empty:
            continue

        split_feature = str(best_split.iloc[0]['feature'])
        split_threshold = float(best_split.iloc[0]['threshold'])
        split_weighted_gini = float(best_split.iloc[0]['weighted_gini'])

        impurity_decrease = node_info['gini'] - split_weighted_gini
        if impurity_decrease < min_impurity_decrease:
            continue

        left_node_id = next_node_id + 1
        right_node_id = next_node_id + 2
        next_node_id += 2

        cur.execute('''
            UPDATE dt_nodes
            SET is_leaf = 0,
                split_feature = ?,
                split_threshold = ?
            WHERE node_id = ?;
        ''', (split_feature, split_threshold, node_id))

        cur.execute('''
            UPDATE dt_node_membership
            SET node_id = (
                SELECT CASE
                    WHEN l.feature_value <= ? THEN ?
                    ELSE ?
                END
                FROM dt_long l
                WHERE l.sample_id = dt_node_membership.sample_id
                  AND l.feature = ?
            )
            WHERE node_id = ?;
        ''', (split_threshold, left_node_id, right_node_id, split_feature, node_

        left_stats = compute_node_stats(left_node_id)
        right_stats = compute_node_stats(right_node_id)

        cur.execute('''
            INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent,
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        ''', (left_node_id, depth + 1, node_id, 'L', 1, None, None, left_stats['

        cur.execute('''
            INSERT INTO dt_nodes(node_id, depth, parent_id, branch_from_parent,
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
        ''', (right_node_id, depth + 1, node_id, 'R', 1, None, None, right_stats

        split_logs.append({
            'node_id': node_id,
            'depth': depth,
            'feature': split_feature,
            'threshold': split_threshold,
            'parent_gini': node_info['gini'],
            'split_weighted_gini': split_weighted_gini,
```

```
            'impurity_decrease': impurity_decrease,
            'left_node_id': left_node_id,
            'left_samples': left_stats['n_samples'],
            'right_node_id': right_node_id,
            'right_samples': right_stats['n_samples']
        })

    conn.commit()

nodes_df = pd.read_sql_query('''
    SELECT *
    FROM dt_nodes
    ORDER BY depth, node_id;
''', conn)

display(nodes_df)

split_log_df = pd.DataFrame(split_logs)
if split_log_df.empty:
    print('No valid splits were found with the current constraints.')
else:
    display(split_log_df)

print(f'max_depth={max_depth}, min_samples_split={min_samples_split}, min_sample
```

	node_id	depth	parent_id	branch_from_parent	is_leaf	split_fea
0	1	0	NaN	None	0	Healthy_life_expecta
1	2	1	1.0	L	0	GDP_per_ca
2	3	1	1.0	R	0	GDP_per_ca
3	4	2	2.0	L	0	Perceptions_of_corrup
4	5	2	2.0	R	0	Generc
5	6	2	3.0	L	0	Social_sup
6	7	2	3.0	R	0	Freedom_to_make_life_chc
7	8	3	4.0	L	1	N
8	9	3	4.0	R	1	N
9	10	3	5.0	L	1	N
10	11	3	5.0	R	1	N
11	12	3	6.0	L	1	N
12	13	3	6.0	R	1	N
13	14	3	7.0	L	1	N
14	15	3	7.0	R	1	N

	node_id	depth	feature	threshold	parent_gini	split_weight
0	1	0	Healthy_life_expectancy	0.469000	0.666666	0.
1	2	1	GDP_per_capita	0.648457	0.239527	0.
2	3	1	GDP_per_capita	1.229000	0.607713	0.
3	4	2	Perceptions_of_corruption	0.179550	0.145161	0.
4	5	2	Generosity	0.136560	0.480151	0.
5	6	2	Social_support	0.777110	0.599114	0.
6	7	2	Freedom_to_make_life_choices	0.498465	0.267716	0.

```
max_depth=3, min_samples_split=30, min_samples_leaf=12, min_impurity_decrease=0.0
001
```

In [8]:
```python
# 3) Evaluate multi-level tree predictions in SQL.
cur.executescript('''
DROP VIEW IF EXISTS dt_predictions;
CREATE VIEW dt_predictions AS
SELECT
    m.sample_id,
    d.score_class AS y_true,
    n.predicted_class AS y_pred,
    n.node_id AS leaf_node_id,
    n.depth AS leaf_depth
FROM dt_node_membership m
JOIN dt_dataset d ON d.sample_id = m.sample_id
JOIN dt_nodes n ON n.node_id = m.node_id
WHERE n.is_leaf = 1;
''')
conn.commit()

tree_eval = pd.read_sql_query('''
SELECT
    AVG(CASE WHEN y_true = y_pred THEN 1.0 ELSE 0.0 END) AS accuracy,
    AVG(CASE WHEN y_true = y_pred THEN 0.0 ELSE 1.0 END) AS zero_one_loss,
    COUNT(*) AS n_samples
FROM dt_predictions;
''', conn)
display(tree_eval)

leaf_summary = pd.read_sql_query('''
SELECT
    n.node_id,
    n.depth,
    n.parent_id,
    n.branch_from_parent,
    n.predicted_class,
    n.n_samples,
    n.gini
FROM dt_nodes n
WHERE n.is_leaf = 1
ORDER BY n.depth, n.node_id;
''', conn)
display(leaf_summary)
```

```
confusion_df = pd.read_sql_query('''
SELECT
    y_true,
    y_pred,
    COUNT(*) AS n
FROM dt_predictions
GROUP BY y_true, y_pred
ORDER BY y_true, y_pred;
''', conn)
display(confusion_df)
```

	accuracy	zero_one_loss	n_samples
0	0.733675	0.266325	781

	node_id	depth	parent_id	branch_from_parent	predicted_class	n_samples	
0	8	3	4	L	low	149	0.101
1	9	3	4	R	low	16	0.429
2	10	3	5	L	low	24	0.000
3	11	3	5	R	middle	22	0.512
4	12	3	6	L	low	50	0.485
5	13	3	6	R	middle	319	0.571
6	14	3	7	L	high	84	0.444
7	15	3	7	R	high	117	0.066

	y_true	y_pred	n
0	high	high	169
1	high	low	1
2	high	middle	91
3	low	low	207
4	low	middle	53
5	middle	high	32
6	middle	low	31
7	middle	middle	197

C) K-means Clustering (k=3, SQL distance + SQL centroid updates)

Features (as required):

- `Score`
- `GDP_per_capita`
- `Social_support`

- `Healthy_life_expectancy`
- `Freedom_to_make_life_choices`
- `Generosity`
- `Perceptions_of_corruption`

Workflow:

1. Min-max normalize all 7 features in SQL
2. Initialize 3 centroids from three score buckets
3. Iterate assignment/update steps (all core math in SQL)
4. Report cluster size, centroid values, and SSE

In [9]:
```
# 1) Prepare normalized points table.
cur.executescript('''
DROP TABLE IF EXISTS kmeans_points;
CREATE TABLE kmeans_points AS
WITH stats AS (
    SELECT
        MIN(Score) AS min_score,
        MAX(Score) AS max_score,
        MIN(GDP_per_capita) AS min_gdp,
        MAX(GDP_per_capita) AS max_gdp,
        MIN(Social_support) AS min_social,
        MAX(Social_support) AS max_social,
        MIN(Healthy_life_expectancy) AS min_health,
        MAX(Healthy_life_expectancy) AS max_health,
        MIN(Freedom_to_make_life_choices) AS min_freedom,
        MAX(Freedom_to_make_life_choices) AS max_freedom,
        MIN(Generosity) AS min_generosity,
        MAX(Generosity) AS max_generosity,
        MIN(Perceptions_of_corruption) AS min_corr,
        MAX(Perceptions_of_corruption) AS max_corr
    FROM happyness
)
SELECT
    rowid AS point_id,
    Country,
    Year,
    (Score - min_score) / NULLIF(max_score - min_score, 0) AS f1_score,
    (GDP_per_capita - min_gdp) / NULLIF(max_gdp - min_gdp, 0) AS f2_gdp,
    (Social_support - min_social) / NULLIF(max_social - min_social, 0) AS f3_soc
    (Healthy_life_expectancy - min_health) / NULLIF(max_health - min_health, 0)
    (Freedom_to_make_life_choices - min_freedom) / NULLIF(max_freedom - min_free
    (Generosity - min_generosity) / NULLIF(max_generosity - min_generosity, 0) A
    (Perceptions_of_corruption - min_corr) / NULLIF(max_corr - min_corr, 0) AS f
FROM happyness, stats;

DROP TABLE IF EXISTS kmeans_centroids;
CREATE TABLE kmeans_centroids (
    cluster_id INTEGER PRIMARY KEY,
    f1_score REAL,
    f2_gdp REAL,
    f3_social REAL,
    f4_health REAL,
    f5_freedom REAL,
    f6_generosity REAL,
    f7_corruption REAL
```

```
);

INSERT INTO kmeans_centroids(cluster_id, f1_score, f2_gdp, f3_social, f4_health,
WITH seeded AS (
    SELECT
        point_id,
        f1_score, f2_gdp, f3_social, f4_health, f5_freedom, f6_generosity, f7_co
        NTILE(3) OVER (ORDER BY f1_score) AS seed_cluster
    FROM kmeans_points
)
SELECT
    seed_cluster AS cluster_id,
    AVG(f1_score),
    AVG(f2_gdp),
    AVG(f3_social),
    AVG(f4_health),
    AVG(f5_freedom),
    AVG(f6_generosity),
    AVG(f7_corruption)
FROM seeded
GROUP BY seed_cluster
ORDER BY seed_cluster;
''')
conn.commit()

display(pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY cluster_id;',
```

	cluster_id	f1_score	f2_gdp	f3_social	f4_health	f5_freedom	f6_generosity	f7
0	1	0.280472	0.283214	0.501152	0.334491	0.454907	0.257932	
1	2	0.527525	0.507107	0.676135	0.570788	0.538167	0.230512	
2	3	0.779383	0.677041	0.791886	0.705512	0.711450	0.294137	

In [10]:
```
# 2) Iterate k-means until convergence or max iterations.
max_iter = 50
tol = 1e-8
history = []

for i in range(max_iter):
    # Assignment step: choose nearest centroid for each point.
    cur.executescript('''
    DROP TABLE IF EXISTS kmeans_assignments;
    CREATE TABLE kmeans_assignments AS
    WITH distances AS (
        SELECT
            p.point_id,
            c.cluster_id,
            (
                POWER(p.f1_score - c.f1_score, 2) +
                POWER(p.f2_gdp - c.f2_gdp, 2) +
                POWER(p.f3_social - c.f3_social, 2) +
                POWER(p.f4_health - c.f4_health, 2) +
                POWER(p.f5_freedom - c.f5_freedom, 2) +
                POWER(p.f6_generosity - c.f6_generosity, 2) +
                POWER(p.f7_corruption - c.f7_corruption, 2)
            ) AS sq_distance,
```

```
            ROW_NUMBER() OVER (
                PARTITION BY p.point_id
                ORDER BY
                    (
                        POWER(p.f1_score - c.f1_score, 2) +
                        POWER(p.f2_gdp - c.f2_gdp, 2) +
                        POWER(p.f3_social - c.f3_social, 2) +
                        POWER(p.f4_health - c.f4_health, 2) +
                        POWER(p.f5_freedom - c.f5_freedom, 2) +
                        POWER(p.f6_generosity - c.f6_generosity, 2) +
                        POWER(p.f7_corruption - c.f7_corruption, 2)
                    ) ASC,
                    c.cluster_id ASC
            ) AS rn
        FROM kmeans_points p
        CROSS JOIN kmeans_centroids c
    )
    SELECT point_id, cluster_id, sq_distance
    FROM distances
    WHERE rn = 1;
    ''')

    old_centroids = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY c

    # Update step: recompute centroids from current assignments.
    cur.executescript('''
    DROP TABLE IF EXISTS kmeans_new_centroids;
    CREATE TABLE kmeans_new_centroids AS
    SELECT
        a.cluster_id,
        AVG(p.f1_score) AS f1_score,
        AVG(p.f2_gdp) AS f2_gdp,
        AVG(p.f3_social) AS f3_social,
        AVG(p.f4_health) AS f4_health,
        AVG(p.f5_freedom) AS f5_freedom,
        AVG(p.f6_generosity) AS f6_generosity,
        AVG(p.f7_corruption) AS f7_corruption
    FROM kmeans_assignments a
    JOIN kmeans_points p ON p.point_id = a.point_id
    GROUP BY a.cluster_id;

    DROP TABLE IF EXISTS kmeans_centroids_next;
    CREATE TABLE kmeans_centroids_next AS
    SELECT
        c.cluster_id,
        COALESCE(n.f1_score, c.f1_score) AS f1_score,
        COALESCE(n.f2_gdp, c.f2_gdp) AS f2_gdp,
        COALESCE(n.f3_social, c.f3_social) AS f3_social,
        COALESCE(n.f4_health, c.f4_health) AS f4_health,
        COALESCE(n.f5_freedom, c.f5_freedom) AS f5_freedom,
        COALESCE(n.f6_generosity, c.f6_generosity) AS f6_generosity,
        COALESCE(n.f7_corruption, c.f7_corruption) AS f7_corruption
    FROM kmeans_centroids c
    LEFT JOIN kmeans_new_centroids n
      ON c.cluster_id = n.cluster_id;

    DELETE FROM kmeans_centroids;
    INSERT INTO kmeans_centroids
    SELECT * FROM kmeans_centroids_next;
    ''')
```

```
    new_centroids = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY c

    merged = old_centroids.merge(new_centroids, on='cluster_id', suffixes=('_old
    movement = 0.0
    for f in ['f1_score', 'f2_gdp', 'f3_social', 'f4_health', 'f5_freedom', 'f6_
        movement += ((merged[f'{f}_old'] - merged[f'{f}_new']) ** 2).sum()

    sse = pd.read_sql_query('SELECT SUM(sq_distance) AS sse FROM kmeans_assignme
    history.append({'iter': i, 'centroid_movement': float(movement), 'sse': floa

    conn.commit()

    if movement < tol:
        break

kmeans_history_df = pd.DataFrame(history)
display(kmeans_history_df.tail(10))
```

	iter	centroid_movement	sse
2	2	0.001476	105.306612
3	3	0.001161	104.706760
4	4	0.000454	104.317727
5	5	0.000055	104.218194
6	6	0.000044	104.201792
7	7	0.000008	104.189082
8	8	0.000008	104.186599
9	9	0.000011	104.183307
10	10	0.000007	104.180035
11	11	0.000000	104.178796

In [11]:
```
# 3) Final clustering outputs: centroid table, cluster sizes, and sample members
final_centroids_df = pd.read_sql_query('SELECT * FROM kmeans_centroids ORDER BY
display(final_centroids_df)

cluster_size_df = pd.read_sql_query('''
SELECT cluster_id, COUNT(*) AS n_points
FROM kmeans_assignments
GROUP BY cluster_id
ORDER BY cluster_id;
''', conn)
display(cluster_size_df)

cluster_samples_df = pd.read_sql_query('''
SELECT
    a.cluster_id,
    p.Country,
    p.Year,
    ROUND(a.sq_distance, 6) AS sq_distance
FROM kmeans_assignments a
JOIN kmeans_points p ON p.point_id = a.point_id
ORDER BY a.cluster_id, a.sq_distance ASC
```

```
LIMIT 30;
''', conn)
display(cluster_samples_df)

final_sse_df = pd.read_sql_query('SELECT SUM(sq_distance) AS total_sse FROM kmea
display(final_sse_df)
```

	cluster_id	f1_score	f2_gdp	f3_social	f4_health	f5_freedom	f6_generosity	f7
0	1	0.293135	0.242214	0.467427	0.285865	0.459903	0.279805	
1	2	0.568943	0.553138	0.717788	0.617485	0.555904	0.210711	
2	3	0.831655	0.743099	0.815880	0.751731	0.792921	0.368550	

	cluster_id	n_points
0	1	247
1	2	394
2	3	140

	cluster_id	Country	Year	sq_distance
0	1	Senegal	2016	0.008529
1	1	Guinea	2019	0.012925
2	1	Guinea	2018	0.027332
3	1	Burkina Faso	2016	0.030192
4	1	Tanzania	2016	0.033146
5	1	Senegal	2015	0.033692
6	1	Sierra Leone	2019	0.035087
7	1	Mauritania	2015	0.035799
8	1	Burkina Faso	2015	0.038016
9	1	Mali	2016	0.038756
10	1	Togo	2019	0.039325
11	1	Ethiopia	2017	0.039634
12	1	Niger	2019	0.041224
13	1	Burkina Faso	2017	0.041910
14	1	Zimbabwe	2016	0.043042
15	1	Yemen	2015	0.043951
16	1	Gambia	2019	0.045160
17	1	Ivory Coast	2019	0.045393
18	1	Guinea	2017	0.045425
19	1	Ghana	2015	0.048837
20	1	Niger	2017	0.050441
21	1	Congo (Brazzaville)	2015	0.050491
22	1	Ethiopia	2018	0.053579
23	1	Niger	2018	0.053585
24	1	Zambia	2016	0.055655
25	1	Burkina Faso	2018	0.058439
26	1	Uganda	2019	0.058572
27	1	Zimbabwe	2017	0.058739
28	1	Mali	2015	0.058763
29	1	Ghana	2017	0.059143

	total_sse
0	104.178796